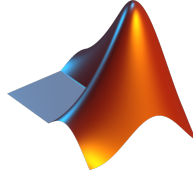


### § Introducción a Matlab para métodos numéricos. §



#### Los métodos numéricos

Consisten en una serie de procedimientos que nos permiten obtener soluciones aproximadas a determinados problemas, cuando no es posible obtener su solución analítica. Estos métodos se apoyan en gran medida en la computación. **Matlab**, que es una herramienta matemática orientada principalmente al cálculo numérico. Sin embargo, en sus versiones más recientes, también incorpora capacidades de cálculo simbólico.

#### Matlab en la actualidad:

Matlab es una de las herramientas más utilizadas en el ámbito académico y profesional para la resolución de problemas de ingeniería y ciencias aplicadas, debido a su capacidad de manejar grandes volúmenes de datos y realizar operaciones complejas de manera eficiente. Además, su integración con múltiples paquetes de simulación, análisis de datos, y generación de informes, lo convierte en una herramienta esencial en diversas áreas del conocimiento, como la física, las matemáticas, la ingeniería eléctrica y la economía, entre otras.

**Nota:** Es recomendable que tengas Matlab instalado antes de avanzar, para que puedas seguir de manera práctica los ejemplos.

#### Instrucciones básicas

Nos permitirán realizar las operaciones fundamentales dentro de la consola de Matlab, también conocida como *Command Window*. Estas instrucciones nos ayudarán a conocer el comportamiento de Matlab y a gestionar el entorno de trabajo, conocido como *Workspace*.

#### Comandos básicos

A continuación, se presenta una lista de comandos esenciales que utilizaremos frecuentemente a lo largo del curso:

- **help** <comando> – Muestra la ayuda detallada del comando especificado.
- **save** <archivo> – Guarda las variables que hay en el *Workspace* dentro del archivo especificado con extensión **.mat**.
- **clear** <variable> – Borra la variable especificada del *Workspace*. Si no se especifica ninguna, se borran todas.
- **clc** – Limpia la consola de comandos, eliminando todo el texto mostrado previamente.
- **whos** – Muestra información detallada sobre las variables que están almacenadas en el *Workspace*.
- **load** <archivo> – Carga en el *Workspace* las variables almacenadas en el archivo con extensión **.mat**.

**Nota:** Estos comandos básicos son esenciales para interactuar con Matlab, ya que permiten gestionar tanto las variables como el entorno de trabajo. Además, facilitan la ejecución y organización de los cálculos.

### Ejemplos de usos

Para ilustrar cómo funcionan estos comandos, a continuación se presentan algunos ejemplos prácticos:

| Comando | Ejemplo                                                         | Resultado                                                              | Descripción                                                 |
|---------|-----------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------|
| help    | » help save                                                     | save - Save workspace variables to file                                | Muestra información sobre el comando <i>save</i> .          |
| save    | » $v = [147 - 1 - 43];$<br>» $w = v.^2;$<br>» save Ejemplo1.mat | Archivo Ejemplo1.mat creado con las variables <i>v</i> y <i>w</i> .    | Guarda variables <i>Workspace</i> en un archivo .mat.       |
| clear   | » clear all                                                     | Elimina todas las variables del <i>Workspace</i> .                     | Borra todas las variables activas en el <i>Workspace</i> .  |
| clc     | » clc                                                           | Limpia la consola.                                                     | Borra todo el contenido mostrado en la consola.             |
| load    | » load Ejemplo1.mat                                             | Las variables <i>v</i> y <i>w</i> se recargan en el <i>Workspace</i> . | Carga las variables almacenadas en el archivo Ejemplo1.mat. |

### Tipos de archivos en Matlab

Matlab utiliza varios tipos de archivos que es necesario conocer. A continuación se describen los más importantes:

- **Archivos .m:** Son archivos de texto que contienen secuencias de comandos que Matlab puede interpretar.
- **Archivos .mat:** Son archivos binarios que almacenan datos.
- **Archivos .fig:** Estos archivos contienen representaciones gráficas generadas en Matlab.
- **Archivos .mlx:** Son archivos de *Live Scripts*, permiten combinar texto formateado, ecuaciones, gráficos y comandos.

### Operaciones aritméticas básica

Matlab permite realizar las operaciones aritméticas básicas de manera sencilla:

- $a + b$  – Suma de *a* y *b*.
- $a - b$  – Resta de *b* a *a*.
- $a * b$  – Multiplicación de *a* y *b*.
- $a / b$  – División de *a* entre *b*.
- $a ^b$  – Eleva *a* a la potencia *b*.

**Nota:** Estas operaciones se pueden aplicar tanto a escalares, como a vectores y matrices, siempre que cumplan con las reglas del álgebra matricial en cuanto a dimensiones.

### Precedencia de Operadores

La **precedencia de operadores** se refiere al orden en que se evalúan los operadores en una expresión matemática o lógica. Cuando una expresión contiene múltiples operadores, la precedencia determina cuáles se deben calcular primero. Por ejemplo:

- **Paréntesis ()** - Tienen la mayor precedencia y se evalúan primero.
- **Exponentes** - Se evalúan después de los paréntesis.

- **Multiplicación y división**  $*$ ,  $/$  - Tienen la misma precedencia y se evalúan de izquierda a derecha.
- **Suma y resta**  $+$ ,  $-$  - Tienen la menor precedencia y se evalúan después de la multiplicación y la división.

#### Ejemplo práctico:

$$\text{result} = 2 + 3 \times 4^2 - \left(\frac{1}{5}\right)$$

La evaluación se lleva a cabo de la siguiente manera:

1. **Paréntesis:** Se evalúa primero  $\left(\frac{1}{5}\right)$  que es 0.2.
2. **Exponentes:** Luego,  $4^2$  que es 16.
3. **Multiplicación:** Después,  $3 \times 16$  que es 48.
4. **Suma y Resta:** Finalmente, se evalúa de izquierda a derecha:  $2 + 48 - 0.2$ , lo que da 49.8.

#### Operaciones con vectores

Supongamos que tenemos un vector  $v$ , y queremos elevar sus elementos al cuadrado de manera individual. Para ello, podemos utilizar el operador  $.$  de Matlab, que realiza la operación elemento a elemento:

```
>> v = [1 2 3 4];
>> w = v.^2;
```

**Nota:** Esto elevará al cuadrado cada elemento del vector  $v$ , produciendo el resultado  $w = [1 \ 4 \ 9 \ 16]$ .

#### Funciones matemáticas en Matlab

Matlab incluye funciones matemáticas predefinidas. A continuación, se listan algunas de las más comunes:

- $\sin(x)$ ,  $\cos(x)$ ,  $\tan(x)$  – Funciones trigonométricas.
- $\log(x)$ ,  $\log_{10}(x)$  – Logaritmo natural y logaritmo en base 10.
- $\exp(x)$  – Función exponencial.

#### Uso de funciones matemáticas

Vamos a calcular una expresión que involucra la función logarítmica:

```
>> x = (log2(8) + 5) / 3;
```

**Nota:** Esto calcula la expresión  $x = \frac{\log_2(8)+5}{3}$  y almacena el resultado en la variable  $x$ .

#### Vectores

Los vectores son conjuntos de valores que se organizan en una dimensión. En Matlab, se pueden definir como vectores fila o como vectores columna:

- **Vector fila:** Se introducen separando sus componentes con espacios o comas.
- **Vector columna:** Se introducen separando sus componentes con punto y coma.

#### Ejemplos de un vector fila y un vector columna:

```
v = [1 2 3 4];    % Vector fila
v = [1; 2; 3; 4]; % Vector columna
```

**Nota:** En Matlab, también es posible transformar un vector fila en un vector columna y viceversa, utilizando el operador de transposición (') o el operador de transposición compleja (.'') para evitar la conjugación.

```
v = [1 2 3 4].';    % Transformar vector fila en vector columna
v = [1; 2; 3; 4].'; % Transformar vector columna en vector fila
```

### Extracción de elementos de un vector

Para extraer uno o varios elementos de un vector, es necesario utilizar índices. En Matlab, los índices comienzan desde 1. Para extraer un solo elemento, indicamos su posición dentro de paréntesis:

```
v = [1 2 3 4]; % Extraer el segundo elemento
elemento = v(2);
```

**Nota:** Si queremos extraer más de un elemento, podemos usar un vector con los índices deseados:

```
v = [1 2 3 4 5 6];
elementos = v([2 4 6]); % Extraer el segundo, cuarto y sexto elemento
```

### Generación de vectores con componentes equiespaciadas

En Matlab, es común generar vectores con componentes equiespaciadas. Existen diferentes formas de hacerlo:

- `a:b` – Genera un vector con componentes espaciadas una unidad desde `a` hasta `b`.
- `a:h:b` – Genera un vector con componentes espaciadas `h` unidades desde `a` hasta `b`.
- `linspace(a,b,n)` – Genera un vector con `n` componentes equiespaciadas entre `a` y `b`.

#### Ejemplos:

```
v = 1:10;           % Vector de 1 a 10
v = 2:2:10;          % Vector de 2 en 2
v = linspace(0,1,100); % Vector de 100 puntos entre 0 y 1
```

### Matrices

Las matrices son arreglos bidimensionales de números organizados en filas y columnas. En Matlab, las matrices se definen utilizando corchetes, separando los elementos de una fila con espacios o comas, y separando las filas con punto y coma.

#### Ejemplo de una matriz $3 \times 3$ :

```
A = [1 -1 0; 0 2 5; 1 3 1];
```

### Extracción de elementos de una matriz

Para extraer un elemento de una matriz, se utiliza la notación (fila, columna). Si se desea extraer más de un elemento, se pueden usar vectores de índices.

#### Ejemplo de extracción de un solo elemento:

```
elemento = A(2,1); % Extraer el elemento de la fila 2, columna 1
```

#### Ejemplo de extracción de varios elementos:

```
elementos = A(2,1:2); % Extraer los elementos de la fila 2, columnas 1 y 2
```

### Matrices especiales

Matlab ofrece funciones predefinidas para crear matrices especiales que son de gran utilidad en diversas aplicaciones matemáticas:

- `zeros(n)` – Crea una matriz cuadrada  $n \times n$  llena de ceros.
- `ones(n)` – Crea una matriz cuadrada  $n \times n$  llena de unos.
- `eye(n)` – Crea una matriz identidad de tamaño  $n$ .

#### Ejemplos:

```
Z = zeros(3); % Matriz de ceros 3x3
O = ones(3);  % Matriz de unos 3x3
I = eye(4);   % Matriz identidad 4x4
```

### Operaciones con matrices

Matlab permite realizar operaciones elementales con matrices, como la suma, resta, multiplicación y división de matrices, así como operaciones más avanzadas como el cálculo del determinante o la matriz inversa.

#### Ejemplo de algunas operaciones:

```
C = A + B;      % Suma de matrices
C = A * B;      % Producto de matrices
A_inv = inv(A); % Inversa de una matriz
det_A = det(A); % Determinante de una matriz
```

### Resolución de sistemas de ecuaciones

Una de las operaciones más comunes con matrices es la resolución de sistemas de ecuaciones lineales. Para un sistema  $Ax = b$ , Matlab permite resolverlo de forma eficiente utilizando el operador contrabarra `\`:

#### Ejemplo:

```
x = A\b; % Resolver Ax = b
```

**Nota:** Este método es más eficiente que calcular explícitamente la inversa de la matriz  $A$ .

#### Ejemplo: Resolución de un sistema de ecuaciones

Vamos a resolver el siguiente sistema de ecuaciones:

$$x_1 + x_2 - 2x_3 = 4$$

$$x_1 - x_2 = 1$$

$$x_1 + 3x_3 = 0$$

Definimos la matriz  $A$  y el vector  $b$  en Matlab:

```
A = [1 1 -2; 1 -1 0; 1 0 3];
b = [4; 1; 0];

x = A\b; % Resolver el sistema
```

### Funciones anónimas

En Matlab, las funciones anónimas son una forma eficiente de definir funciones rápidas y sencillas sin necesidad de almacenarlas en un archivo `.m` por separado. Estas funciones se almacenan en el *Workspace* y pueden ser utilizadas para realizar cálculos rápidos sobre una o más variables.

### Definición de funciones anónimas

La sintaxis básica para definir una función anónima es la siguiente:

```
f = @(x) expresión;
```

**Nota:** Aquí, el símbolo @ indica que estamos definiendo una función anónima, x es el parámetro de entrada, y expresión es la operación que la función realiza. Las funciones anónimas son útiles para funciones matemáticas simples, como polinomios o expresiones algebraicas.

#### Ejemplo: Definición de una función simple

Definamos la función  $f(x) = e^{-x} + \frac{x^2}{\sin(\pi x)}$  en Matlab como una función anónima

```
\begin{verbatim}
f = @(x) exp(-x) + x^2 / sin(pi*x);
```

Ahora, podemos evaluar esta función en cualquier valor de x, o incluso en un vector de valores:

```
resultado = f(1);           % Evaluar en x = 1
valores = [1 2 3 4];
resultado_vectores = f(valores); % Evaluar en un vector de valores
```

**Nota:** Este ejemplo muestra cómo las funciones anónimas pueden operar tanto en escalares como en vectores de manera sencilla.

#### Funciones de varias variables

Las funciones anónimas también permiten el uso de múltiples variables. La sintaxis es similar, pero se deben definir todas las variables dentro del paréntesis.

```
f = @(x, y) expresión;
```

#### Ejemplo: Función de dos variables

Definamos la función  $f(x, y) = e^{-x} \cdot (\cos(\pi y) + \sin(\pi y))$  en Matlab:

```
f = @(x, y) exp(-x) * (cos(pi*y) + sin(pi*y));
```

Podemos evaluar esta función en valores específicos para x y y:

```
resultado = f(1, 2); % Evaluar en x = 1, y = 2
```

#### Ventajas de las funciones anónimas

Las funciones anónimas ofrecen varias ventajas:

- Son rápidas de definir y usar, especialmente para operaciones sencillas.
- No requieren un archivo .m separado, ya que se almacenan directamente en el *Workspace*.
- Se pueden usar en combinaciones con otras funciones, como las funciones de optimización o integración numérica.

#### Ejemplo: Funciones anónimas y optimización

Deseamos encontrar el valor mínimo de la función  $f(x) = x^3 - 3x + 1$  en el intervalo  $[-2, 2]$ . Podemos definir la función como una función anónima y luego utilizar la función `fminbnd` para encontrar el mínimo:

```
f = @(x) x^3 - 3*x + 1;
[min_x, min_f] = fminbnd(f, -2, 2);
```

**Nota:** Este código buscará el valor de x que minimiza la función dentro del intervalo especificado.

#### Uso de funciones anónimas con múltiples operaciones

Las funciones anónimas ofrecen varias ventajas: Las funciones anónimas pueden incluir varias operaciones y llamadas a otras funciones de Matlab.

**Ejemplo:**

Supongamos que queremos definir una función que calcule la media y la desviación estándar de un conjunto de valores:

```
stats = @(x) [mean(x), std(x)];
```

Si aplicamos esta función a un vector de valores:

```
valores = [1, 2, 3, 4, 5];  
resultado = stats(valores);
```

El vector `resultado` contendrá la media y la desviación estándar del vector `valores`.

**Funciones anónimas dentro de otras funciones**

Las funciones anónimas pueden usarse como argumentos de otras funciones. Esto es especialmente útil cuando se trabaja con funciones de orden superior, como `integral`, `fzero` o `fminbnd`.

**Ejemplo: Integración de una función anónima**

Supongamos que queremos integrar la función  $f(x) = \frac{1}{1+x^2}$  en el intervalo  $[0, 1]$ . Podemos definir la función como una función anónima y utilizar la función `integral` de Matlab para calcular la integral:

```
f = @(x) 1 ./ (1 + x.^2);  
resultado = integral(f, 0, 1);
```

**Nota:** Este código calculará el valor de la integral definida de la función.

**Archivos .m**

Los archivos con extensión `.m` son ficheros de texto que contienen código de Matlab. Matlab puede interpretar estos archivos como comandos o scripts. Hay dos tipos principales de archivos `.m`:

**Scripts**

Un **script** es un conjunto de líneas de código que se ejecutan secuencialmente, como si las estuvas escribiendo directamente en la consola. Los scripts no requieren parámetros de entrada ni de salida. Además, operan sobre las variables que ya existen en el *Workspace* y pueden modificarlas.

**Ejecución de un script**

Para ejecutar un script en Matlab, solo necesitas guardarlo con la extensión `.m` y luego llamarlo por su nombre (sin la extensión). Si, por ejemplo, el archivo se llama `mi_script.m`, simplemente puedes ejecutarlo en la consola de Matlab con:

```
>> mi_script
```

**Ejemplo de un script**

A continuación se muestra un ejemplo de un script que resuelve un sistema de ecuaciones lineales utilizando la eliminación de Gauss:

```
A = [1 1 -2; 1 -1 0; 1 0 3];  
b = [4; 1; 0];  
x = A\b; % Resuelve el sistema Ax = b  
disp('La solución del sistema es:');  
disp(x); % disp muestra texto o valores en la ventana de comandos.
```

## Funciones

Una **función** en Matlab es un bloque de código que se define en un archivo `.m` y acepta parámetros de entrada y salida. A diferencia de los scripts, las funciones trabajan dentro de su propio *Workspace* local, lo que significa que las variables que definas dentro de la función no interfieren con las variables en el *Workspace* principal, a menos que se especifique lo contrario a través de parámetros de salida.

La estructura básica de una función en Matlab es la siguiente:

```
function [salida1, salida2] = nombre_funcion(entrada1, entrada2)
    % Comentarios sobre la función
    % Código de la función
end
```

### Ejemplo de una función

A continuación, se muestra un ejemplo de una función que resuelve un S.E.L. usando la eliminación de Gauss. La función acepta dos parámetros de entrada, la matriz **A** y el vector **b**, y devuelve el vector solución **x**.

```
function x = resolver_sistema(A, b)
    % La función x = resolver_sistema(A, b)
    % resuelve el sistema de ecuaciones lineales Ax = b
    x = A\b;
end
```

Este archivo de función debe guardarse con el nombre `resolver_sistema.m`. Luego, puedes llamar a la función desde la consola de Matlab o desde otro script, proporcionando los parámetros necesarios.

```
A = [1 1 -2; 1 -1 0; 1 0 3];
b = [4; 1; 0];
x = resolver_sistema(A, b);
disp(x);
```

## Diferencias entre scripts y funciones

Aunque tanto los scripts como las funciones se guardan en archivos `.m`, existen diferencias importantes entre ambos:

- **Variables:** Los scripts utilizan las variables del *Workspace* global, mientras que las funciones crean un *Workspace* local que no afecta las variables fuera de la función.
- **Entradas y salidas:** Las funciones pueden aceptar parámetros de entrada y devolver valores de salida, mientras que los scripts no permiten parámetros de entrada ni salida de forma explícita.
- **Reutilización:** Las funciones son más flexibles que los scripts, ya que pueden ser reutilizadas fácilmente con diferentes entradas sin modificar el código.

## Buenas prácticas al trabajar con archivos `.m`

Al trabajar con archivos `.m`, es importante seguir algunas buenas prácticas para asegurarse de que el código sea claro y reutilizable:

- **Comentarios:** Siempre es recomendable documentar tu código con comentarios.
- **Nombres descriptivos:** Utiliza nombres descriptivos tanto para los scripts como las variables.
- **Funciones modulares:** Si una función en tu código es compleja, considera dividirla en funciones más pequeñas.
- **Evitar scripts largos:** Considerar crear una función para tareas más específicas.

**Nota:** Esté es un ejemplo donde utilizamos `linsolve` en lugar de la notación de barra invertida (`\`) para resolver sistemas de ecuaciones lineales. Es útil cuando deseas mayor control sobre la solución, como cuando trabajas con matrices grandes o necesitas especificar opciones adicionales.



**Ejemplo: Reutilización de funciones**

```

A1 = [1 2 3; 4 5 6; 7 8 10];      A2 = [2 1 -1; -3 -1 2; -2 1 2];
b1 = [6; 15; 25];                b2 = [8; -11; -3];

% Aquí podemos definir opciones para linsolve si lo deseamos
opts.UT = true; % Indica que A es triangular superior, si corresponde

x1 = resolver_sistema(A1, b1, opts);    opts.UT = false; % Cambiamos para el segundo sistema
disp('Solución para el sistema 1:');    x2 = resolver_sistema(A2, b2, opts);
disp(x1);                               disp('Solución para el sistema 2:');
                                         disp(x2);

```

**Nota:** Este código reutiliza la función `resolver_sistema` para resolver dos sistemas de ecuaciones diferentes, mostrando cómo las funciones permiten que el código sea más eficiente y reutilizable.

**Estructuras de Control**

Las estructuras de control son fundamentales en cualquier lenguaje de programación, ya que permiten dirigir el flujo de ejecución de un programa en función de condiciones y repeticiones. En Matlab, disponemos de varias estructuras de control que nos permiten implementar lógica condicional y bucles para iteraciones.

**Estructuras Condicionales**

Las estructuras condicionales permiten que un programa tome decisiones y ejecute diferentes bloques de código en función de ciertas condiciones.

**if, elseif, else**

La estructura `if` evalúa una condición y ejecuta un bloque de código si la condición es verdadera. Podemos extender esta estructura con `elseif` y `else` para manejar múltiples condiciones. La sintaxis básica es:

```

if condición1      % Código si condición1 es verdadera
elseif condición2 % Código si condición2 es verdadera
else               % Código si ninguna condición anterior es verdadera
end

```

**Ejemplo: Determinar si un número es positivo, negativo o cero**

```

numero = input('Ingrese un número: ');

if numero > 0
    disp('El número es positivo.');
```

```

else numero < 0
    disp('El número es negativo.');
```

```

end

```

**Operadores Relacionales y Lógicos**

Para formular condiciones, es esencial conocer los operadores relacionales y lógicos que ofrece Matlab.

**Ejemplo con operadores lógicos: Determinar si un número está dentro de un rango específico**

```

numero = 5;

if numero >= 1 && numero <= 10
    disp('El número está entre 1 y 10.');
```

```

else
    disp('El número está fuera del rango.');
```

```

end

```

- Operadores lógicos:

- & : Y lógico (AND)
- | : O lógico (OR)
- ~ : Negación lógica (NOT)

- Operadores relacionales:

- == : Igual a
- ~= : No igual a
- > : Mayor que
- < : Menor que
- >= : Mayor o igual que
- <= : Menor o igual que

### switch, case, otherwise

La estructura **switch** es útil cuando se necesita seleccionar entre múltiples opciones posibles basadas en el valor de una variable o expresión. La sintaxis básica es:

```
switch expresión
    case valor1    % Código si expresión == valor1
    case valor2    % Código si expresión == valor2
    otherwise      % Código si expresión no coincide con ningún caso anterior
end
```

### Ejemplo: Programa que asigna un mensaje según el día de la semana

```
dia = 'Lunes';
switch dia
    case 'Lunes'
        disp('Inicio de semana.');
```

```
    case 'Viernes'
        disp('Fin de semana cercano.');
```

```
    case {'Sábado', 'Domingo'}
        disp('Es fin de semana.');
```

```
    otherwise
        disp('Día entre semana.');
```

```
end
```

### Estructuras Iterativas

Las estructuras iterativas o bucles permiten repetir la ejecución de un bloque de código múltiples veces, ya sea un número específico de veces o mientras se cumpla una condición.

### for Loop

El bucle **for** se utiliza cuando se conoce el número de iteraciones que se desea realizar. Sintaxis básica:

```
for variable = inicio:incremento:fin % Código a ejecutar en cada iteración
end
```

### Ejemplo: Calcular y mostrar los primeros 10 números factoriales

```
factorial = 1;
for n = 1:10
    factorial = factorial * n;
end
```

### while Loop

El bucle **while** se utiliza cuando el número de iteraciones no está definido y depende de una condición lógica:

```
while condición % Código a ejecutar mientras la condición sea verdadera
```

**Ejemplo: Encontrar el  $<$  num. entero  $n$  tal que la suma de los números de 1 a  $n$  sea  $>$  que 1000**

```

suma = 0;
n = 0;
while suma <= 1000
    n = n + 1;
    suma = suma + n;
end

```

**break y continue**

Dentro de los bucles, podemos utilizar las sentencias `break` y `continue` para controlar el flujo de la iteración.

**Ejemplo con break: Buscar el primer número divisible por 7 en un rango de números**

```

for n = 1:100
    if mod(n, 7) == 0
        fprintf('El primer número divisible por 7 es %d\n', n);
        break; % Sale inmediatamente del bucle en el que se encuentra.
    end
end

```

**Ejemplo con continue: Mostrar solo los números impares entre 1 y 10**

```

for n = 1:10
    if mod(n, 2) == 0
        % continue; Salta a la siguiente iteración, omitiendo el código en la iteración actual.
        continue; % Salta los números pares
    else
        fprintf('%d es impar\n', n);
    end
end

```

**Anidamiento de Estructuras de Control**

Es posible anidar estructuras de control, es decir, colocar una dentro de otra, para manejar lógica más compleja.

**Ejemplo: Generar una matriz identidad de tamaño  $n$  sin utilizar la función `eye`**

```

n = 5;
I = zeros(n); % Inicializa una matriz de ceros de tamaño n x n

for i = 1:n
    for j = 1:n
        if i == j
            I(i,j) = 1;
        end
    end
end

disp('Matriz identidad generada:');
disp(I);

```

**Consideraciones sobre Eficiencia y Buenas Prácticas**

- **Prealocación de variables:** Preasignar el tamaño de las matrices o vectores antes de utilizarlos.
- **Evitar bucles innecesarios:** Utiliza operaciones sobre vectores y matrices en lugar de bucles.
- **Comentarios y legibilidad:** Añade comentarios y utiliza sangrías adecuadas.

**Nota:** MATLAB está optimizado para manejar operaciones en matrices y vectores, es la razón por la cual las operaciones vectorizadas son más rápidas que los bucles for.

#### Ejemplo de optimización:

Comparación entre un bucle for y una operación vectorizada para elevar al cuadrado los elementos de un vector

```
vec = 1:1000;
resultado = zeros(size(vec)); % size obtiene las dimensiones de una matriz o un arreglo.

% Utilizando función for
for i = 1:length(vec)
    resultado(i) = vec(i)^2;
end

% Usando operaciones vectorizadas
vec = 1:1000;
resultado = vec.^2;
```

#### Representación de funciones de una variable

La representación de gráficos en Matlab es `plot`, que se utiliza para dibujar gráficos en dos dimensiones (2D), `plot` toma como argumentos los valores del eje  $x$  y los valores del eje  $y$  que se desean graficar.

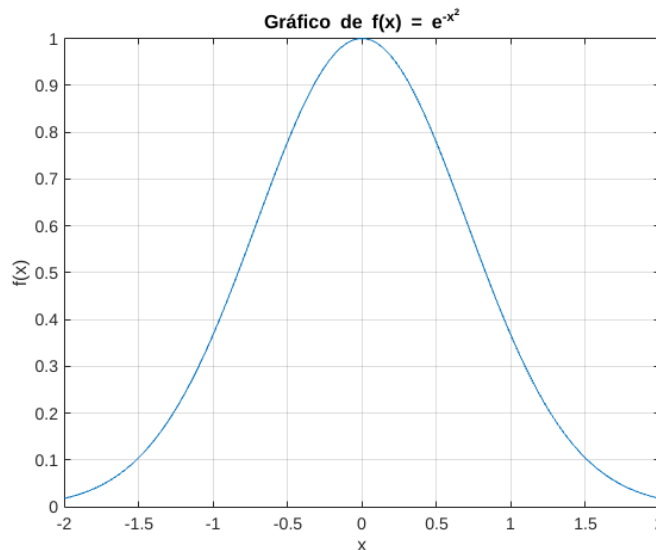
**Sintaxis básica:**

```
plot(x, y);
```

#### Ejemplo: Representar la función $f(x) = e^{-x^2}$

Supongamos que queremos representar la función  $f(x) = e^{-x^2}$  para  $x$  en el intervalo  $[-2, 2]$ :

```
x = linspace(-2, 2, 100); % Generar 100 puntos entre -2 y 2
f = exp(-x.^2);           % Calcular los valores de f(x)
plot(x, f);               % Graficar la función
title('Gráfico de f(x) = e^{-x^2}'); % Añadir título
xlabel('x');              % Etiqueta del eje x
ylabel('f(x)');           % Etiqueta del eje y
grid on;                 % Añadir cuadrícula al gráfico
```



## Personalización de gráficos

Matlab permite personalizar la apariencia de los gráficos de múltiples formas, como cambiando los colores, el estilo de las líneas, el tipo de marcas, entre otros.

### Sintaxis para personalizar el gráfico:

```
plot(x, y, 'estilo');
```

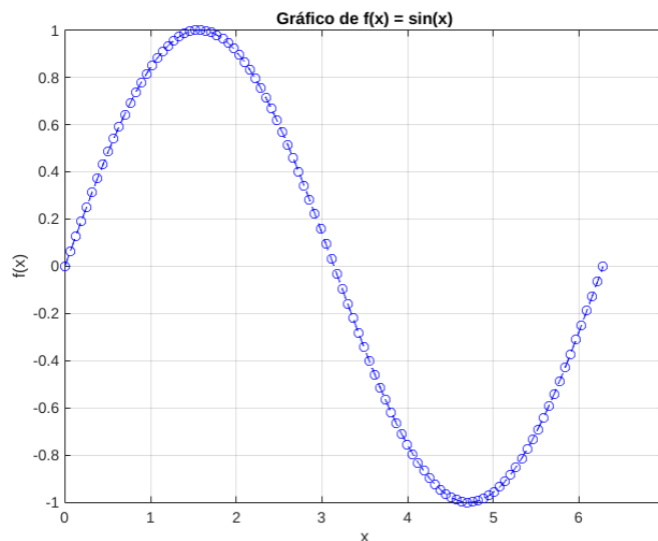
Donde 'estilo' puede incluir el color, el tipo de línea y los marcadores de los puntos, como se muestra a continuación:

- 'r': Rojo (color de la línea)
- 'o': Marcador en forma de círculo
- '-': Línea discontinua

### Ejemplo: Representar la función $f(x) = \sin(x)$

Representar la función  $f(x) = \sin(x)$  con una línea discontinua azul y círculos en los puntos de la gráfica:

```
x = linspace(0, 2*pi, 100);    % Generar puntos entre 0 y 2pi
y = sin(x);                    % Calcular valores de sin(x)
plot(x, y, 'b-o');              % Graficar con línea azul discontinua y círculos
title('Gráfico de f(x) = sin(x)');
xlabel('x');
ylabel('f(x)');
grid on;
```



## Representación de funciones de dos variables

Para representar funciones de dos variables en Matlab, se utilizan las funciones `mesh`, `surf`, o `plot3`, que permiten visualizar superficies en tres dimensiones (3D). Para ello, primero se necesita crear un "mallado" o grid de puntos en el plano  $xy$  con la función `meshgrid`.

### Sintaxis básica:

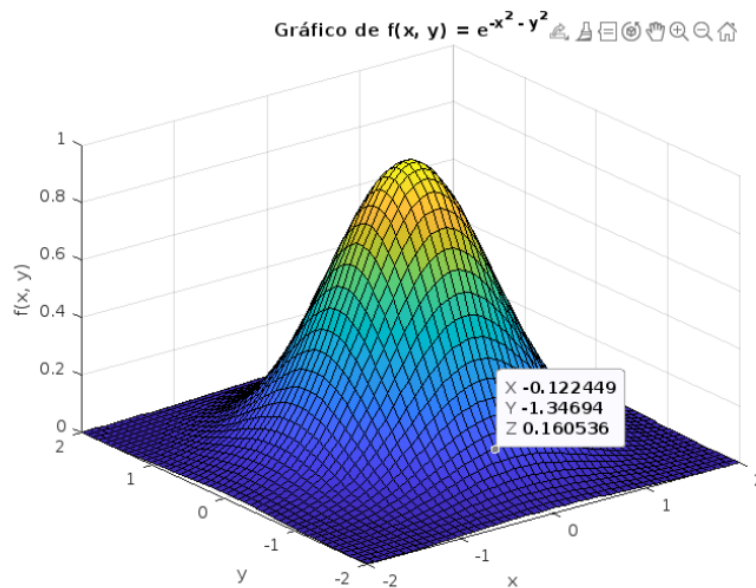
```
[X, Y] = meshgrid(x, y);
Z = f(X, Y);
surf(X, Y, Z); % Representar la superficie
```

### Ejemplo: Representar la función $f(x, y) = e^{-x^2-y^2}$

Vamos a representar la función  $f(x, y) = e^{-x^2-y^2}$  en el intervalo  $[-2, 2] \times [-2, 2]$ :

```
x = linspace(-2, 2, 50); % Generar puntos entre -2 y 2
y = linspace(-2, 2, 50);
[X, Y] = meshgrid(x, y); % Crear la malla de puntos
Z = exp(-X.^2 - Y.^2);    % Calcular los valores de Z
surf(X, Y, Z);           % Graficar la superficie

title('Gráfico de f(x, y) = e^{-x^2 - y^2}');
xlabel('x');
ylabel('y');
zlabel('f(x, y)');
```



### Tipos de gráficos en 3D

Matlab proporciona diferentes tipos de gráficos tridimensionales, según el tipo de visualización que desees:

- `plot3(X, Y, Z)`: Dibuja una gráfica en 3D conectando puntos.
- `mesh(X, Y, Z)`: Genera una superficie en forma de malla (sin relleno).
- `surf(X, Y, Z)`: Genera una superficie con relleno, coloreada en función de la altura.

### Subgráficos

En Matlab, se pueden combinar múltiples gráficos en una misma ventana utilizando la función `subplot`. Esto permite dividir la figura en una cuadrícula de subgráficos.

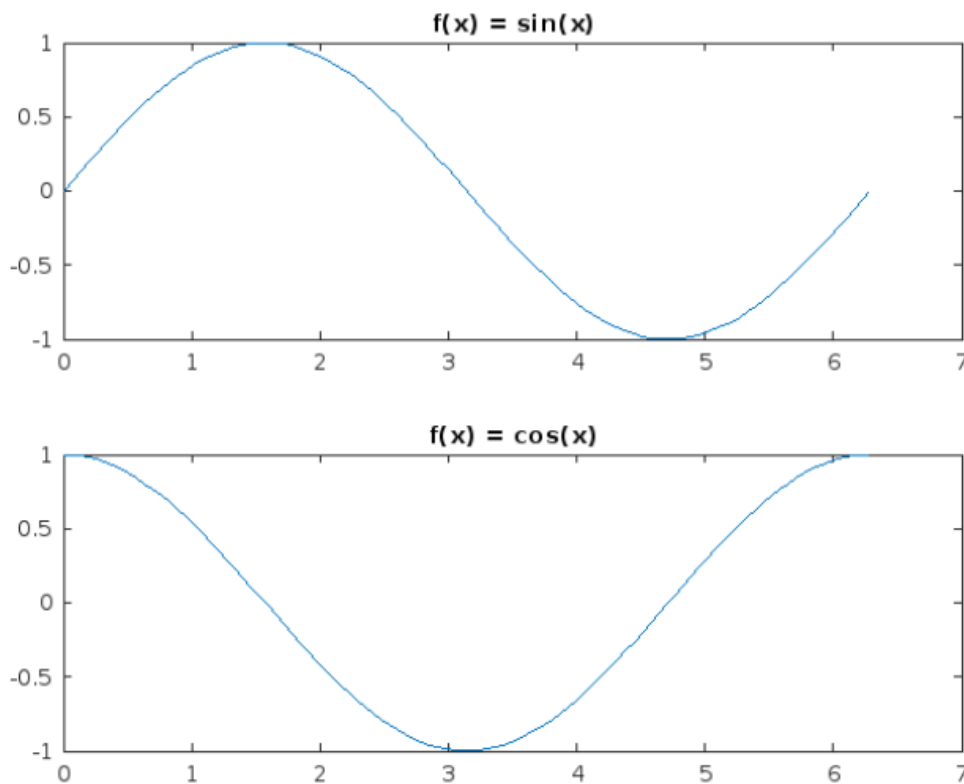
**Sintaxis básica:**

```
subplot(m, n, p);
```

**Nota:** Donde *m* es el número de filas, *n* es el número de columnas, y *p* es el índice del subgráfico actual.

**Ejemplo: Dibujar en una misma ventana los gráficos de las funciones  $\sin(x)$  y  $\cos(x)$** 

```
x = linspace(0, 2*pi, 100);  
y1 = sin(x);  
y2 = cos(x);  
  
subplot(2, 1, 1); % Primer gráfico en la primera fila  
plot(x, y1);  
title('f(x) = sin(x)');  
  
subplot(2, 1, 2); % Segundo gráfico en la segunda fila  
plot(x, y2);  
title('f(x) = cos(x)');
```

**Guardar gráficos**

Una vez que se ha generado un gráfico, Matlab permite guardarlo en diferentes formatos, como .jpg, .png, .pdf, entre otros. Esto se puede hacer de manera programática utilizando la función `saveas`.

**Ejemplo: Guardar el gráfico actual como un archivo .png**

```
saveas(gcf, 'grafico.png');
```

**Cálculo Simbólico**

Matlab también tiene capacidades de cálculo simbólico mediante el uso de la herramienta **Symbolic Math Toolbox**. El cálculo simbólico es útil cuando se desea trabajar con expresiones algebraicas, derivadas, integrales y otros cálculos que no necesariamente requieren una evaluación numérica inmediata.

### Declaración de variables simbólicas

Primero debemos declarar las variables simbólicas utilizando la función `syms`. Esto le indica a Matlab que las variables serán tratadas de manera simbólica, permitiendo realizar operaciones algebraicas exactas.

#### Sintaxis:

```
syms variable1 variable2 ... variablen;
```

#### Ejemplo: Declarar dos variables simbólicas

```
syms x y;
```

**Nota:** Las variables `x` y `y` estarán listas para realizar operaciones simbólicas como derivadas, integrales, etc.

### Operaciones algebraicas simbólicas

Las variables simbólicas permiten realizar operaciones algebraicas exactas, como simplificación, expansión y factorización de expresiones.

#### Ejemplos de operaciones algebraicas simbólicas

##### Expansión de una expresión:

```
expr = (x + 2)*(x - 3);  
expand(expr) % Esto dará como resultado  $x^2 - x - 6$ .
```

##### Factorización de una expresión:

```
expr = x^2 - x - 6;  
factor(expr) % Esto devolverá  $(x + 2)(x - 3)$ .
```

##### Simplificación de expresiones:

```
expr = (x^2 - 4)/(x - 2);  
simplify(expr) % Matlab simplificará esta expresión a  $x + 2$ .
```

### Derivadas simbólicas

Una de las funcionalidades más útiles del cálculo simbólico es la capacidad de calcular derivadas de manera simbólica. La función `diff` se utiliza para calcular la derivada de una función.

#### Sintaxis:

```
diff(expr, var, n)
```

**Nota:** Donde `expr` es la expresión simbólica, `var` es la variable respecto a la cual se deriva, y `n` es el orden de la derivada (opcional; por defecto es 1).

#### Ejemplo: Calcular la derivada de la función $f(x) = x^3 - 3x + 5$

```
syms x;  
f = x^3 - 3*x + 5;  
derivada_f = diff(f, x);
```

**Nota:** Esto devolverá la derivada simbólica  $f'(x) = 3x^2 - 3$ .



## Integrales simbólicas

Otra operación común es la integración simbólica. Matlab permite calcular integrales indefinidas y definidas utilizando la función `int`.

### Sintaxis:

```
int(expr, var)           % Integral indefinida
int(expr, var, a, b)     % Integral definida
```

### Ejemplo: Calcular la integral indefinida de la función $f(x) = e^{-x^2}$

```
syms x;
f = exp(-x^2);
integral_f = int(f, x);
```

**Nota:** Devuelve la integral indefinida en términos simbólicos, ya que no tiene una solución analítica exacta.

### Ejemplo: Calcular una integral definida

Supongamos que queremos la integral de  $f(x) = x^2$  en el intervalo  $[0, 2]$ :

```
syms x;
f = x^2;
integral_definida = int(f, x, 0, 2);
```

**Nota:** El resultado será  $\frac{8}{3}$ .

## Ecuaciones simbólicas

Matlab también permite resolver ecuaciones algebraicas y diferenciales de manera simbólica. Para ello, utilizamos la función `solve`, que puede resolver ecuaciones lineales, polinómicas y diferenciales.

### Sintaxis:

```
solve(expr, var)
```

### Ejemplo: Resolver la ecuación cuadrática $x^2 - 5x + 6 = 0$

```
syms x;
sol = solve(x^2 - 5*x + 6 == 0, x);
```

**Nota:** Matlab devolverá las raíces de la ecuación,  $x = 2$  y  $x = 3$ .

## Ecuaciones diferenciales simbólicas

Para resolver ecuaciones diferenciales simbólicas, Matlab utiliza la función `dsolve`. Esta función permite resolver tanto ecuaciones diferenciales ordinarias como sistemas de ecuaciones diferenciales.

### Sintaxis:

```
dsolve(eq, var)
```

### Ejemplo: Resolver la ecuación diferencial $\frac{dy}{dx} = 3x^2$

```
syms x y(x);
eq = diff(y, x) == 3*x^2;
sol = dsolve(eq);
```

**Nota:** La solución será  $y(x) = x^3 + C$ , donde  $C$  es la constante de integración.

### Cálculo de límites

Los límites también se pueden calcular simbólicamente con la función `limit`. Matlab permite calcular límites laterales y límites en puntos infinitos.

**Sintaxis:**

```
limit(expr, var, punto)
```

**Ejemplo:** Calcular el límite de la función  $f(x) = \frac{\sin(x)}{x}$  cuando  $x$  tiende a 0

```
syms x;  
f = sin(x)/x;  
limite = limit(f, x, 0);
```

**Nota:** El resultado será 1, que es el valor del límite.

### Cálculo de series de Taylor

Matlab también permite calcular la expansión en series de Taylor de una función utilizando la función `taylor`.

**Sintaxis:**

```
taylor(expr, var, punto, orden)
```

**Ejemplo:** Expansión en series de Taylor de la función  $\sin(x)$  alrededor de  $x = 0$  hasta orden 5

```
syms x;  
taylor_sin = taylor(sin(x), x, 0, 'Order', 5);
```

**Nota:** El resultado será  $x - \frac{x^3}{6}$ , que es la aproximación de  $\sin(x)$  hasta el tercer grado.